B.Tech. BCSE497J - Project-I

OBJECT-CENTRIC EVENT LOG MINING FOR ACADEMIC CAPSTONE PROCESSES: A NATIVE OCEL 2.0 APPROACH

Submitted in partial fulfillment of the requirements for the degree of

Bachelor of Technology

in

Computer Science and Engineering

by

23BCE0434 MITISHA SHARMA

23BCE0498 TANMAY KHAMBETE

23BCE0500 AAKAASH SATHYA SARAVANAN

Under the Supervision of

Dr. Kalyanaraman P.

Professor Grade 1

School of Computer Science and Engineering (SCOPE)



VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

October 2026

DECLARATION

We hereby declare that the project entitled **Object-Centric Event Log Mining for Academic Capstone Processes: A Native OCEL 2.0 Approach** submitted by us, for the award of the degree of *Bachelor of Technology in Computer Science and Engineering* to VIT is a record of bonafide work carried out by us under the supervision of Dr. Kalyanaraman P.

We further declare that the work reported in this project has not been submitted and will not be submitted, either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university.

Place : Vellore

Date :

Signature: _____ **Mitisha Sharma**

Signature: _____ **Tanmay Khambete**

Signature: _____ **Aakaash Sathya Saravanan**

CERTIFICATE

This is to certify that the project entitled **Object-Centric Event Log Mining for Academic Capstone Processes: A Native OCEL 2.0 Approach** submitted by **Mitisha Sharma (23BCE0434), Tanmay Khambete (23BCE0498) and Aakaash Sathya Saravanan (23BCE0500)**, School of Computer Science and Engineering, VIT, for the award of the degree of *Bachelor of Technology in Computer Science and Engineering*, is a record of bonafide work carried out by them under my supervision during Fall Semester 2026-2027, as per the VIT code of academic and research ethics.

The contents of this report have not been submitted and will not be submitted either in part or in full, for the award of any other degree or diploma in this institute or any other institute or university. The project fulfills the requirements and regulations of the University and in my opinion meets the necessary standards for submission.

Place : Vellore

Date :

Signature of the Guide

Examiner(s)

Dr. Boominathan P

Computer Science and Engineering

ACKNOWLEDGEMENTS

We are deeply grateful to the management of Vellore Institute of Technology (VIT) for giving us the opportunity and the resources to take up this project. Their commitment to a conducive learning environment has been important to our academic journey, and the support and infrastructure provided by VIT let us explore and develop our ideas.

Our sincere thanks to Dr. Jaisankar N, the Dean of the School of Computer Science and Engineering (SCOPE), for the Dean's unwavering support and encouragement. The Dean's leadership and vision have inspired us to strive for excellence, and the dedication to academic excellence and innovation has been a constant source of motivation. We appreciate the effort that goes into creating an environment that nurtures creativity and critical thinking.

We express our profound appreciation to Dr. Boominathan P, the Head of the Department of Computer Science and Engineering, for their insightful guidance and continuous support. Their expertise and advice shaped the direction of our project, and the collaborative, supportive atmosphere they keep made learning easier for us.

We are immensely thankful to our project guide, Dr. Kalyanaraman P., for their dedicated mentorship and valuable feedback. Their patience, knowledge and encouragement carried us through this project, and the time they spent sharing expertise and giving thoughtful guidance helped us refine our ideas and methods.

Thank you all for your contributions and support.

Mitisha Sharma
Tanmay Khambete
Aakaash Sathya Saravanan

TABLE OF CONTENTS

Sl.No	Contents	Page No.
	Abstract	**x**
1.	**INTRODUCTION**	**1**
	1.1 Background	1
	1.2 Motivations	2
	1.3 Scope of the Project	2
2.	**PROJECT DESCRIPTION AND GOALS**	**3**
	2.1 Literature Review	3
	2.2 Research Gap	4
	2.3 Objectives	4
	2.4 Problem Statement	4
	2.5 Project Plan	5
3.	**TECHNICAL SPECIFICATION**	**6**
	3.1 Requirements	6
	3.1.1 Functional	6
	3.1.2 Non-Functional	6
	3.2 Feasibility Study	6
	3.2.1 Technical Feasibility	6
	3.2.2 Economic Feasibility	7
	3.2.3 Social Feasibility	7
	3.3 System Specification	7
	3.3.1 Hardware Specification	7
	3.3.2 Software Specification	8
4.	**DESIGN APPROACH AND DETAILS**	**9**
	4.1 System Architecture	9
	4.2 Design	9
	4.2.1 Data Flow Diagram	9
	4.2.2 Use Case Diagram	10
	4.2.3 Class Diagram	11
	4.2.4 Sequence Diagram	12
	4.3 OCEL 2.0 Model of the Capstone Lifecycle (Design Only)	12
5.	**METHODOLOGY AND TESTING**	**14**
	5.1 Module Description	14
	5.1.1 M1: Acquire and Verify	14
	5.1.2 M2: Convert and Load	14
	5.1.3 M3: Distortion Metrics	15
	5.1.4 M4: Consequence Analysis	15
	5.1.5 M5: Predicate Engine (Appendix-Level)	16
	5.1.6 M6: Validation	17

Sl.No	Contents	Page No.
	5.1.7 M7: Summaries	17
	5.2 Testing	17
6.	**PROJECT DEMONSTRATION**	**19**
	6.1 Step 1: Check that the data is present	19
	6.2 Step 2: Run the self-check	19
	6.3 Step 3: Distortion of the Enron log	19
	6.4 Step 4: Consequences for the Enron log	20
	6.5 Step 5: The cross-log summary	20
7.	**RESULT AND DISCUSSION**	**21**
	7.1 The Seven Logs	21
	7.2 Distortion by Flattening	21
	7.3 Consequences for Standard Outputs	23
	7.4 Process Discovery on Flat Logs	25
	7.5 Discussion	26
	7.5.1 Threats to Validity	27
	7.6 Cost Analysis	27
8.	**CONCLUSION**	**28**
	8.1 Limitations	28
	8.2 Future Work	29
9.	**REFERENCES**	**30**
	APPENDIX A - SAMPLE CODE	**32**

LIST OF FIGURES

Figure No.	Title	Page No.
2.1	Project calendar, August to December 2026 (grey bars are not part of the delivered work)	5
4.1	System architecture of the flattening study	9
4.2	Data flow diagram, level 0	10
4.3	Data flow diagram, level 1	10
4.4	Use case diagram	11
4.5	Class diagram of the main structures	11
4.6	Sequence diagram of one end-to-end run	12
4.7	Object types, relations and events of the capstone lifecycle	13
7.1	Duplication and dropped events for the seven logs, by regime	22
7.2	Object-type pairs that share more than half of their objects across cases	22
7.3	Activity ranking agreement: duplication alone versus the whole process	24
7.4	Entity ranking agreement from a denormalised flat export	24
7.5	Spurious directly-follows edges, worst case notion per log	25
7.6	Replay fitness against the share of the process's activities in the discovered model, one dot per case notion	26

LIST OF TABLES

Table No.	Title	Page No.
1.1	Scope of the project	2
2.1	Summary of the reviewed literature	3
2.2	Work packages and status	5
3.1	Functional requirements	6
3.2	Non-functional requirements	6
3.3	Cost items	7
3.4	Hardware used	7
3.5	Software used	8
4.1	Object types of the capstone model	13
5.1	Sources of the seven logs	14
5.2	Distortion metrics for a case notion T	15
5.3	Verification checks	17
6.1	Demonstration steps	19
7.1	Profile of the seven logs	21
7.2	Flattening distortion per log (analysed case notions)	21
7.3	Consequences of flattening per log	23
7.4	Process discovery on the flat logs (Inductive Miner, noise threshold 0.2)	25
7.5	Cost of producing and reproducing the study	27

LIST OF ABBREVIATIONS

AGPL	GNU Affero General Public Licence
AoE2	Age of Empires II
CC BY	Creative Commons Attribution
COOC	Cross-Object Observability Coverage
CROP	Code Review Open Platform (Gerrit code-review dataset)
CSV	Comma-Separated Values
DF	Directly-Follows
DFD	Data Flow Diagram
E2O	Event-to-Object
HTTP	Hypertext Transfer Protocol
JSON	JavaScript Object Notation
MD5	Message-Digest Algorithm 5
MSR	Mining Software Repositories
O2O	Object-to-Object
OCEL	Object-Centric Event Log
OULAD	Open University Learning Analytics Dataset
SCOPE	School of Computer Science and Engineering
SDG	Sustainable Development Goal
SHA-256	Secure Hash Algorithm, 256-bit
SQL	Structured Query Language
TRL	Technology Readiness Level
UCI	University of California, Irvine
UML	Unified Modeling Language
VIT	Vellore Institute of Technology
XML	Extensible Markup Language

SYMBOLS AND NOTATIONS

T	Object type chosen as the case notion
U	Another object type, the entities being ranked or nested inside T
δ(T)	Deficiency: share of events with no T-object
κ(T)	Convergence share: share of covered events related to two or more T-objects
c(T)	Copies per event: flat rows divided by covered events
ρ	Spearman rank correlation coefficient

ABSTRACT

When a student leader submits a report on a capstone portal, that single event concerns a project, a team, three students, a guide and a report version. Process mining, however, requires every event to belong to exactly one case. When the event is flattened to one case notion, it can be copied into several cases (convergence), dropped altogether (deficiency) or merged with repeated events that belong to other objects (divergence). Object-centric event logs, in the OCEL 2.0 standard, keep the many-to-many links instead.

We began this project to study capstone review processes with OCEL 2.0, but portal logs could not be obtained. We also found no public capstone or thesis-management event log. We therefore narrowed our claims. The capstone is presented as a documented OCEL 2.0 design, and we measured flattening on seven real public logs: Gerrit code review (CROP), Open University learning analytics (OULAD), Angular commits, German Bundestag procedures, Enron e-mail, an Ethereum window and Age of Empires II matches. No synthetic or simulated data was used anywhere.

We built a reproducible pipeline that downloads and checks each source, converts the Gerrit and OULAD data to OCEL 2.0, computes flattening distortion for 38 case notions, and compares what an analyst would read off a flat view against the object-centric one: activity rankings, directly-follows edges, process variants, entity rankings and the process model a miner discovers.

The logs fall into three groups. In star logs (CROP, OULAD) every event touches one object of each type, so flattening copies nothing. In partitioned logs (Bundestag, Age of Empires II, Ethereum) one case notion drops 69% to 91% of the events in the median. In fan-out logs (Angular, Enron) flattening does duplicate events, up to 81.65 copies of one event in Angular. Duplication alone rarely changes an activity ranking, since only 2 of 38 case notions fall below a rank correlation of 0.95. The bigger effect is narrowing: in 20 of 38 case notions the flat log shows fewer than half of the process's activities, and 2 of the 15 case notions that cover at least half of their log are among them. A model discovered from a narrowed flat log still reaches a replay fitness of 0.955 or more, so the usual quality check gives no warning. Counting rows in a denormalised flat export can also invert an entity ranking, with worst-case rank correlations of 0.16 (Enron), -0.01 (Ethereum) and -0.41 (Age of Empires II).

We checked parts of the pipeline with an independent SQL implementation, a brute-force self-check, an independent recomputation of the OULAD counts and byte-identical re-runs. A plain-Python re-implementation reproduces the distortion metrics exactly on all seven full logs. Those checks caught three bugs of our own, which we fixed. No one outside the team has labelled the findings, and the capstone model is still a design that has not met portal data. The measurement work is under review at the Journal of Intelligent Information Systems.

CHAPTER 1
INTRODUCTION

1.1 Background

Consider what happens when a student leader submits a capstone report. That single event concerns the project, the team, three students, the guide who will verify it and the report version being submitted. A process-mining tool, however, needs to know which one of these is the case.

This question motivates the project. Classical process mining starts from an event log in which every event has a case identifier, an activity name and a timestamp, and the techniques built on top of it (discovery, conformance checking, bottleneck analysis) assume that each event belongs to exactly one case.

If the project is chosen as the case, each project's history stays together, but the guide's workload across all projects disappears. If the student is chosen, the same submission appears three times, once per team member. A useful comparison is a group photograph. A flat view cuts one person out of the picture. Every person can be cut out in turn, but the resulting set of cut-outs no longer shows who was standing next to whom.

The literature names these distortions. When one event is copied into several cases, the result is *convergence*. When repeated events that belong to different objects of another type are merged into one case, the result is *divergence* [1]. Events that have no object of the chosen type are dropped from the flat log altogether, which is called *deficiency*.

Object-centric event logs avoid this choice. One event can refer to any number of objects, each with a type, and objects can also be linked to each other. OCEL 2.0 [2] standardises this structure, with JSON, XML and SQLite formats, a qualifier on every link, and object attributes that can change over time.

Several studies show why this structure matters. Keeping the object graph improves machine-learning tasks on process data compared with flattened features [3], and Adams and van der Aalst [4] address convergence, divergence and deficiency together. One-to-many links produced false directly-follows relations in a food-company case study [5], and object-centric data has quality problems of its own [6]. In education, Miri et al. [7] converted Moodle activity into OCEL and found that group and resource links remain invisible in a student-centric log.

What we could not find was a study that measures these effects across several real logs. The problem is widely acknowledged, but its size has not been reported. Suitable public data does exist: the Open University Learning Analytics Dataset (OULAD) [8] covers students, courses and assessments, and CROP [9] links Gerrit code reviews to code changes, a process closely related to report review.

A capstone has the same structure. Hassani et al. [10] describe a department supervising more than 250 final-year computing projects, and Premchaiswadi and Porouhan [11] mined an online peer-review process and found bottlenecks and rule violations in reviewer handling. The Educational Capstone Portal, whose architecture is documented in our project repository, handles registration, team formation, report compilation, three review stages, faculty decisions and scoring, and it writes an audit-log row for every state change. Its process is therefore

strongly object-centric.

We set out to study that process directly, but portal logs could not be obtained, and we found no public capstone or thesis-management event log. We therefore narrowed what we claim. The capstone is presented as a documented OCEL 2.0 design, and the measurements are made on seven real public logs [16]. This report keeps the two parts clearly separate.

1.2 Motivations

First, flattening happens without notice. Tools apply it by default and rarely report how many events were copied, dropped or merged.

Second, the evidence is thin. Convergence and divergence are usually explained with small examples or a single case study, and we found no work that reports how often and how strongly they occur across several real logs.

Third, capstones are coordination work. The questions that matter in practice (whether one guide is overloaded, or which team resubmits repeatedly) cut across cases, so a case-centric view does not fit them.

Fourth, our claims are limited to the data we can access. Portal data is not available to us, so a re-runnable measurement on public logs is the strongest evidence we can provide.

1.3 Scope of the Project

Table 1.1 states what the project covers and what it does not, so that its limits are explicit from the start.

Table 1.1: Scope of the project

In scope	Out of scope
An OCEL 2.0 model of the capstone lifecycle, designed from the portal documentation	Any claim about how real capstone teams behave; no portal data was used
A re-runnable pipeline: download, verify, convert, load, analyse	Predictive monitoring on flat versus object-centric logs
Flattening distortion for 38 case notions on seven real public logs	Independent human labelling of the findings by a second rater
A consequence analysis: activity rankings, directly-follows edges, variants, entity rankings and discovered models, flat versus object-centric	Integrating a portal adapter; this is future work
Independent checks of our own code, and a documented list of bugs they found	Synthetic or simulated data of any kind

Two further points apply. A rule-based engine from our earlier work is kept at appendix level, since eleven rules on two logs cannot support a conclusion. In addition, a Moodle log from the University of Cádiz was downloaded as an eighth source but is not used in any result.

The report is organised as follows. Chapter 2 covers the literature and the plan, Chapter 3 the requirements and feasibility, Chapter 4 the design, Chapter 5 the modules and testing, Chapter 6 the demonstration, Chapter 7 the results and cost, Chapter 8 the conclusion and Chapter 9 the references. Appendix A contains sample code.

CHAPTER 2
PROJECT DESCRIPTION AND GOALS

2.1 Literature Review

The literature falls into four groups: the flattening problem itself, the OCEL format, what flattening does to results and to data quality, and the education and capstone papers. Table 2.1 sums up the ones that matter most for our questions. The full list is in Chapter 9.

Table 2.1: Summary of the reviewed literature

Ref.	Objective	Method	Strength and limitation	Finding
van der Aalst (2019) [1]	Name the problems of one-case-per-event logs	Conceptual analysis with worked examples	Defines convergence and divergence. No measurement on real logs	Flattening copies events across cases or mis-orders them
Berti et al. (2024) [2]	A standard exchange format for object-centric data	OCEL 2.0 specification: JSON, XML, SQLite, qualifiers, O2O links	Expressive, with tool support. Extraction still needs domain decisions	One event can relate to many typed objects
Adams, Park, van der Aalst (2023) [3]	Keep object-centric graph structure for machine learning	Compare graph encodings with flattened features on predictive tasks	Shows the value of structure. Focus is prediction, not analysis outputs	Structure-preserving encodings are favoured over flattening
Waibel et al. (2022) [5]	Causal structure from relational data	Causal event graph on one food-company case	Avoids false edges. Only one case study	One-to-many and many-to-many links give spurious directly-follows relations
Kabierski et al. (2026) [6]	Classify quality issues in object-centric data	Classification checked on real logs and practitioner interviews	Practitioner-validated. Classifies issues but gives no distortion rates	Object-centric data has quality problems of its own, e.g. wrong object relations
Miri et al. (2026) [7]	Object-centric educational process mining	Moodle to OCEL conversion, real course case study	First educational use. One course, not a transactional lifecycle	Group and resource links are invisible in student-centric logs
Premchaiswadi, Porouhan (2015) [11]	Mine a peer-review process in an online system	Event log from review handling; discovery and conformance	Real review workflow. One case notion	Bottlenecks and rule violations found in reviewer handling
Hassani et al. (2018) [10]	Manage more than 250 computing final-year projects	Case study of supervision, panels and rubrics	Real capstone practice. No event data	Capstones need coordinated supervision, submission and assessment

On the flattening problem itself, van der Aalst [1] explained why one-case-per-event logs misrepresent processes with several interacting entities, and Adams and van der Aalst [4] went on to cover convergence, divergence and deficiency together. Both papers are conceptual, and neither reports how often these effects occur in real data.

OCEL 2.0 [2] is the format we build on. It stores events, objects, event-to-object (E2O) links and object-to-object (O2O) links, each with qualifiers, in JSON, XML or SQLite. Our own converters write the SQLite form, and we read the JSON and XML forms of the public logs.

On consequences, Waibel et al. [5] used a causal event graph to show that one-to-many links produce false directly-follows edges, but only on one company's data. Kabierski et al. [6] classified the quality issues of object-centric data; they describe the kinds of problems but do not count them. Adams et al. [3] showed that keeping the object graph helps predictive tasks.

In education, Miri et al. [7] is the closest related work, since they applied object-centric mining

to Moodle data. This project is therefore not the first use of OCEL in education. Premchaiswadi and Porouhan [11] mined an online peer-review process with a single case notion, and Hassani et al. [10] describe how a department supervises a large number of final-year projects, with no event data.

2.2 Research Gap

Taken together, the reviewed work leaves three gaps.

First, flattening problems are named but not measured. Convergence and divergence appear in small or single-case examples, and most object-centric evidence we found uses simulated logs.

Second, work on education and capstones relies on learning-platform data. Object-centric educational mining uses Moodle activity. A review lifecycle with submit, review, reject, resubmit and score has no public object-centric dataset.

Third, capstone portal logs are not available. Searches of Zenodo, 4TU and the web found no public capstone or thesis-management event log, so we cannot test the capstone claim directly.

Our approach is therefore to measure flattening on real review-like and multi-entity logs, to give the capstone an OCEL 2.0 model, and to state the limits explicitly.

2.3 Objectives

The project has five objectives, in the order we worked on them.

- O1. Model the capstone lifecycle as OCEL 2.0 from the documented portal behaviour.
- O2. Build a reproducible pipeline: download, verify, convert, load and analyse.
- O3. Measure flattening distortion on seven real logs without hand-picked rules.
- O4. Compare standard outputs (activity rankings, directly-follows edges, variants, entity counts and discovered process models) between flat and object-centric views.
- O5. Verify every result with independent checks and keep the code re-runnable.

The work touches two of the United Nations Sustainable Development Goals: Goal 4, Quality Education, since it concerns how academic review processes are recorded and understood, and Goal 9, Industry, Innovation and Infrastructure, since it adds open tooling for object-centric process data. On the Technology Readiness Level scale we put the pipeline at TRL 4 (validated in the laboratory, on seven public logs) and portal integration at TRL 2 to 3. Those levels are our own estimate.

2.4 Problem Statement

We have an object-centric event log with several object types, and we pick one type as the case notion. Two questions follow. How much does the flat log differ from the object-centric one, that is, how many events are copied, dropped or merged? And do those differences change what an analyst reads from the standard outputs: activity rankings, directly-follows edges, process variants, entity rankings and discovered models? Both questions must be answered on real data only, with checks that do not depend on our own code, and the capstone lifecycle must be specified in OCEL 2.0 even though no portal data exists.

2.5 Project Plan

Figure 2.1 shows the project calendar. At Review 2 we presented a prototype that was evaluated on synthetic data. The team then decided to use real data only, so that prototype and its results are excluded from this work, and the plan below reflects the work done after that decision.

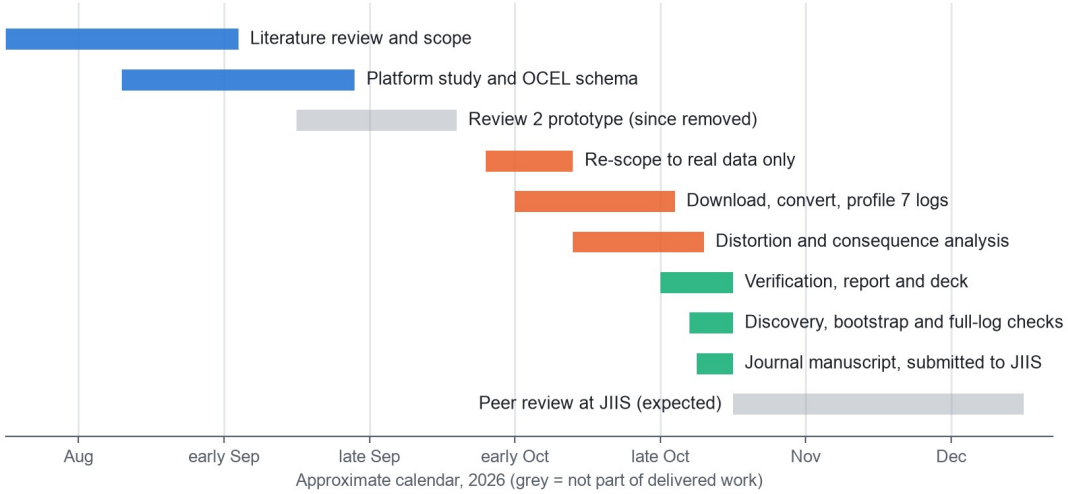


Figure 2.1: Project calendar, August to December 2026 (grey bars are not part of the delivered work)

Table 2.2: Work packages and status

Work package	Content	Status
WP1	Literature review and scope	Completed
WP2	Platform study and OCEL 2.0 schema of the capstone lifecycle	Completed (design only)
WP3	Re-scope to real data only; locate and download public logs	Completed
WP4	Convert CROP and OULAD to OCEL 2.0; profile all seven logs	Completed
WP5	Distortion metrics and consequence analysis	Completed
WP6	Independent verification, report and presentation	Completed
WP7	Process-discovery comparison, bootstrap intervals and full-log verification	Completed
WP8	Journal manuscript	Submitted to the Journal of Intelligent Information Systems (Springer), October 2026; under review
WP9	Independent labelling of the findings	Planned

CHAPTER 3
TECHNICAL SPECIFICATION

3.1 Requirements

3.1.1 Functional

Table 3.1 lists the functional requirements and the module that covers each one. Chapter 5 describes the modules.

Table 3.1: Functional requirements

ID	The system shall	Module
FR1	Download datasets from a manifest; check size and MD5 where the publisher provides them, and record a SHA-256 for every file	M1
FR2	Extract archives without trusting member paths	M1
FR3	Convert Gerrit code reviews (CROP) and OULAD into OCEL 2.0 SQLite	M2
FR4	Load OCEL from SQLite, JSON, XML and bundled formats, with hashed identifiers for personal data	M2
FR5	Compute flattening distortion for every object type of every log	M3
FR6	Compare flat and object-centric outputs: rankings, edges, variants and entities	M4
FR7	Validate results with independent implementations	M6
FR8	Export CSV, JSON and Markdown tables; re-run the whole study with one command	M7

3.1.2 Non-Functional

Table 3.2: Non-functional requirements

ID	Requirement	How we meet or check it
NFR1	Reproducibility: the same input gives the same bytes	Tracked result files were byte-identical after a re-run
NFR2	Privacy: personal identifiers are not published	E-mail addresses and names are hashed on load; no VIT student data is used
NFR3	Resource limit: runs on an ordinary laptop	Developed on an 8 GiB machine; the Ethereum log is analysed as a 300,000-event window
NFR4	Licensing: only openly licensed data	All seven logs are CC BY 4.0; the licence is recorded per source
NFR5	Integrity and safety of data handling	Size and MD5 checked against the Zenodo record; SHA-256 recorded for all files; archives with unsafe member paths rejected
NFR6	Transparency of limits	Every result table states what was and was not checked

3.2 Feasibility Study

3.2.1 Technical Feasibility

The pipeline is written in Python with open-source libraries and runs on a single laptop. The only memory constraint was the Ethereum log, whose 3.2 million events do not fit in 8 GiB of memory, so we analyse a contiguous window of 300,000 events and state this wherever that log appears. For Enron, the distortion analysis took about 9.5 seconds and the consequence analysis

about 4. Run time was not a risk; correctness was, which is why Chapter 5 devotes most of its space to verification.

3.2.2 Economic Feasibility

No data, software or cloud time was purchased. Table 3.3 lists the items. The only cost that cannot be quantified is the team's own time.

Table 3.3: Cost items

Item	Source	Incremental cost
Datasets (seven logs)	Zenodo and UCI repositories, CC BY 4.0	Nil
Analysis software	Python and open-source libraries (see Section 3.3.2)	Nil
Hardware	A laptop the team already owned	Nil
Storage	About 1.3 GB of downloaded and extracted files and 132 MB of derived OCEL files	Nil
Cloud or paid services	None used	Nil

One licensing point needs attention. The `pm4py` library is released under AGPL-3.0, and we use it as a reference implementation of flattening and for process discovery. The code is public on GitHub and is released under AGPL-3.0, which is compatible with `pm4py`.

3.2.3 Social Feasibility

Two logs contain data about people. Enron has real e-mail addresses, and CROP has developer names that its authors had already pseudonymised. We hash identifiers again when a log loads and never print raw values. No VIT student, guide or portal record was used. The method lets anyone who analyses multi-entity workflows, including academic review, see in advance what a flat view will hide.

3.3 System Specification

3.3.1 Hardware Specification

Table 3.4: Hardware used

Item	Specification
Machine	Apple silicon laptop (arm64)
Memory	8 GiB
Storage used	1.3 GB raw (downloaded and extracted), 132 MB derived OCEL files; the one-time download is about 350 MB

3.3.2 Software Specification

Table 3.5: Software used

Component	Version	Use
Python	3.14.5	All analysis code
pandas, NumPy, SciPy	3.1.0, 2.5.4, 1.18.1	Tables, vectorised metrics, Spearman correlation
lxml	6.1.3	XML OCEL loading
pm4py	2.7.23.11	Reference flattening implementation (AGPL-3.0)
DuckDB	2.0.0	Reads the parquet OCEL bundles (Enron, Ethereum) without extra libraries
Matplotlib	3.11.2	Figures
SQLite	3.54.0	OCEL 2.0 storage and the independent SQL cross-check
macOS, Git, Make	-	Operating system, version control, one-command re-run

CHAPTER 4
DESIGN APPROACH AND DETAILS

4.1 System Architecture

Figure 4.1 reads from left to right. The seven public logs enter on the left. The first stage downloads each file, checks it against the publisher's record where one exists, and records a SHA-256 checksum. A second stage extracts archives safely. Then the conversion stage turns CROP and OULAD into OCEL 2.0, and every log ends up in the same four tables: events, objects, event-to-object links and object-to-object links.

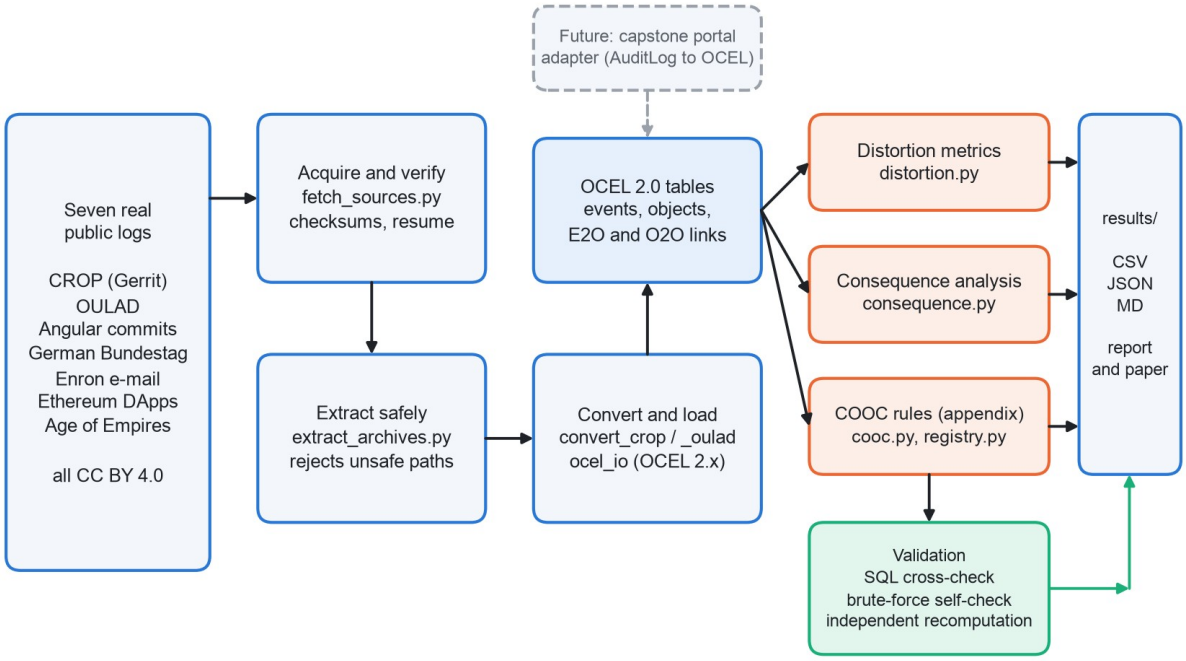


Figure 4.1: System architecture of the flattening study

Three engines read those tables. The distortion engine computes deficiency, convergence, divergence and coupling for each case notion. The consequence engine compares flat and object-centric outputs. The rule-based engine is the appendix-level part. A validation stage re-implements the key computations independently, and all results are written as CSV, JSON and Markdown files from which the report and the paper are produced. The dashed box at the top has not been built. It marks where an adapter for the portal's audit log would connect, should de-identified logs be approved.

4.2 Design

4.2.1 Data Flow Diagram

Figure 4.2 is the level-0 diagram, with one process and two external entities. Public data publishers supply dataset archives and licences. The researchers and the guide supply run commands and settings, and receive tables, figures and validation status.

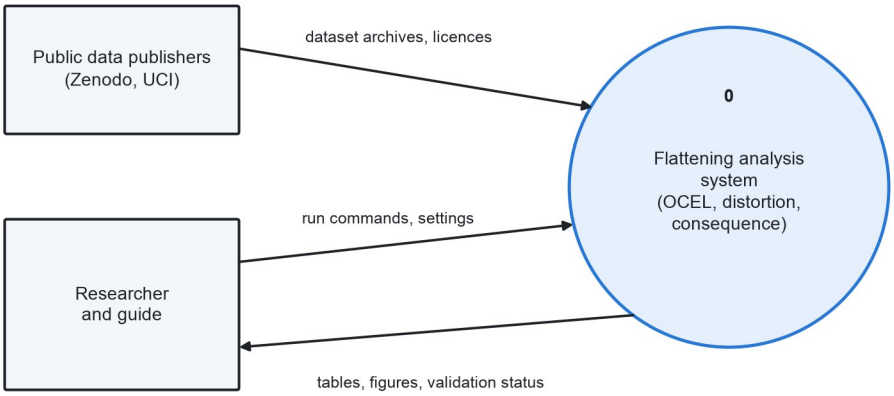


Figure 4.2: Data flow diagram, level 0

Figure 4.3 decomposes that process into six sub-processes. Process 1.0 acquires and verifies archives into store D1. Process 2.0 extracts and converts them into the OCEL logs of store D2. Processes 3.0 (distortion metrics) and 4.0 (consequence analysis) read D2 and write the metric tables of store D3. Process 5.0 validates, and process 6.0 summarises the results for the researchers.

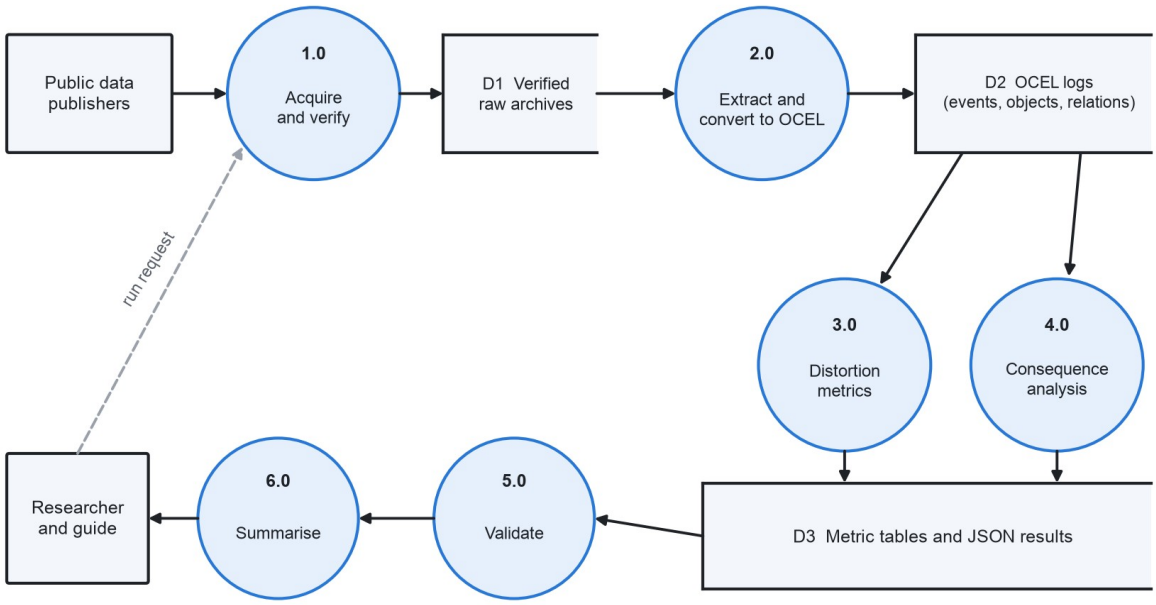


Figure 4.3: Data flow diagram, level 1

4.2.2 Use Case Diagram

Figure 4.4 has two actors. The researcher (the team) downloads and verifies datasets, converts a source to OCEL 2.0, computes distortion metrics, runs the consequence analysis, validates against independent code and exports tables. The guide or panel reviews the results and approves them.

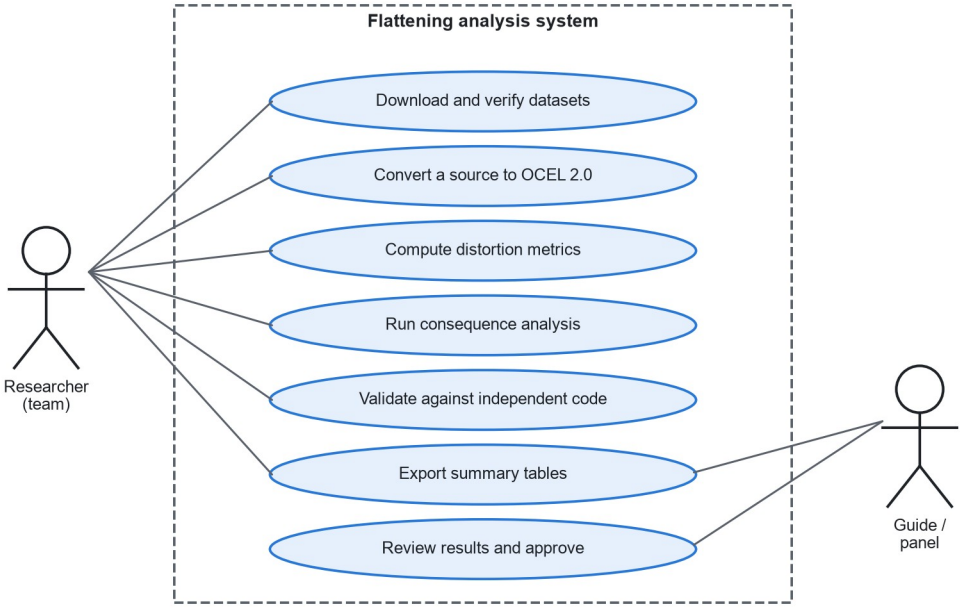


Figure 4.4: Use case diagram

4.2.3 Class Diagram

Figure 4.5 shows the main data structures. `Ocel` holds the four tables of a loaded log. `Prepared` wraps an `Ocel` with integer-coded arrays, which makes the rankings and directly-follows counts fast to compute. `View` and `Predicate` belong to the rule-based engine, where a predicate is evaluated on a view of the log (native, flat or denormalised flat). Note that in the code the loader and the registry are modules of functions, not classes; they are drawn as classes to make their roles clear.

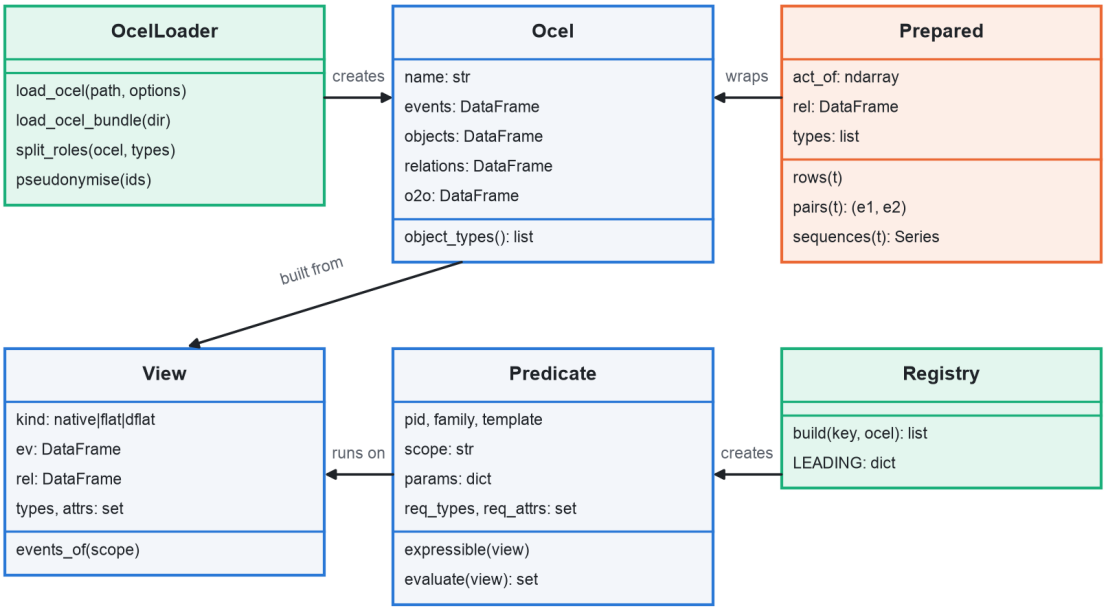


Figure 4.5: Class diagram of the main structures

4.2.4 Sequence Diagram

Figure 4.6 follows one full run. The researcher starts the download, which verifies and records each file. Conversion writes OCEL 2.0 SQLite. The distortion engine asks the loader for an `Ocel` and writes its JSON results, and the consequence engine does the same. The self-check recomputes the directly-follows pairs and the entity ranking in plain Python, and the summaries come last.

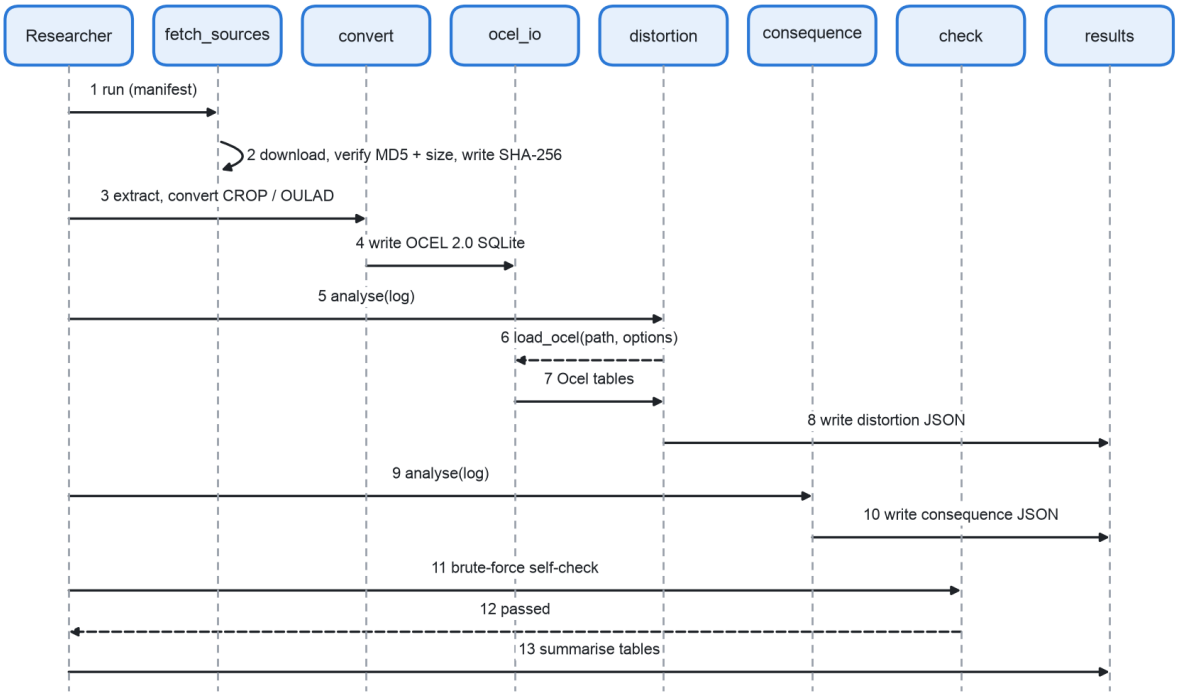


Figure 4.6: Sequence diagram of one end-to-end run

4.3 OCEL 2.0 Model of the Capstone Lifecycle (Design Only)

This section describes the model we designed for the portal, and it is a design, not a result. We derived it from the portal's documentation (the audit-trail, submission-system and scoring-system documents in the repository), and it has not been populated with portal data because none is available.

Table 4.1: Object types of the capstone model

Object type	Meaning	Example relations (qualifiers)
Student	A team member; the leader may submit	member-of (team)
Team	Group executing one project	executes (project)
Project	The capstone project	has-report, guides
Faculty (guide)	Guide who approves ideas, opens reviews and decides	guides (project), reviews (stage)
Report	The report for a project	has-version
Report version	One compiled version; incremented on each rejection	has-score
Review stage	R1, R2 or R3	at-stage
Score	Similarity and AI percentages; written once, then locked	has-score (from version)

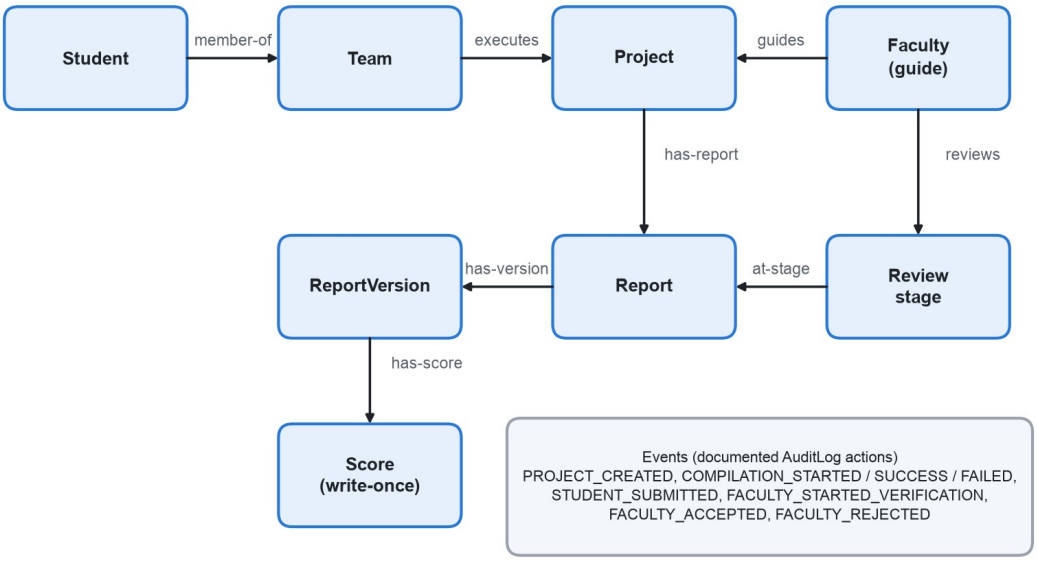


Figure 4.7: Object types, relations and events of the capstone lifecycle

The portal writes an audit-log row for every state transition, and the documented action vocabulary gives eight event types: `PROJECT_CREATED`, `COMPILATION_STARTED`, `COMPILATION_SUCCESS`, `COMPILATION_FAILED`, `STUDENT_SUBMITTED`, `FACULTY_STARTED_VERIFICATION`, `FACULTY_ACCEPTED` and `FACULTY_REJECTED`. Each row carries a project identifier, the acting student or faculty member and a timestamp. In the model, an event relates to its project, to the acting student or faculty member, and, through joins on the project, to the team, the report version and the stage.

A few documented facts shape the model. A score is written once and then locked. A rejection uses up one of three attempts and increments the report version. Compilation failures are free retries and do not use an attempt. The documentation lists the automatic move from stage R1 to R2 to R3 as a known gap, so the stage relation is part of the design, but the current system does not fully populate it.

CHAPTER 5
METHODOLOGY AND TESTING

5.1 Module Description

There are seven modules, M1 to M7. Each one is a small Python script in the `realdata/src` directory, and each reads what the one before it wrote. Figure 4.1 shows how they connect.

5.1.1 M1: Acquire and Verify

`fetch_sources.py` reads a manifest (`sources.json`) that lists each source with its record number, licence and expected files. For a Zenodo record it asks Zenodo for the file list, downloads each file, and compares the size and MD5 checksum with the record. A mismatch deletes the file and stops the run. OULAD comes from the UCI mirror and has no published checksum, so there we cap the download size and record the SHA-256. Every file gets a SHA-256 in a `SOURCE.md` next to it. If a download stalls, the script resumes from the partial file with an HTTP Range request; this was needed once during the study.

`extract_archives.py` unpacks each archive into a new, empty directory next to it, and it refuses any archive with an absolute path or a parent-relative (`..`) member path.

Table 5.1 lists the sources, and Table 7.1 has the sizes of the logs.

Table 5.1: Sources of the seven logs

Log	Publisher and record	Authors of the dataset
CROP (Gerrit)	Zenodo 3599150	Paixao, Krinke, Han, Harman
OULAD	UCI repository, DOI 10.24432/C5KK69	Kuzilek, Hlosta, Zdrahal
Angular commits	Zenodo 8430332	Pegoraro
German Bundestag (20th period, filtered)	Zenodo 16811928	Diers, Peeva, Rennert, van der Aalst
Enron e-mail (top 100,000 mails)	Zenodo 21989732	Berti
Ethereum DApps (300,000-event window)	Zenodo 21989580	Berti
Age of Empires II (10-match filter)	Zenodo 13365584	Liss, Elbert, Flath

5.1.2 M2: Convert and Load

Five logs already come in an OCEL format: Angular and Bundestag as JSON, Age of Empires as SQLite, Enron and Ethereum as parquet bundles. For the other two, `convert_crop.py` and `convert_oulad.py` build OCEL 2.0 SQLite files. CROP becomes three object types (change, person, project) with events like upload, comment, approve, merge and abandon. OULAD becomes four (student, enrollment, presentation, assessment) with events like register, submit and unregister.

`ocel_io.py` loads every format into the same four pandas tables. It hashes personal identifiers with a salted hash (the Enron e-mail addresses and the CROP names), splits the Enron e-mail address type by role (sender, recipient, copied) so each role can be a case notion, and for Ethereum cuts a contiguous window of 300,000 events. The flattening function used throughout

the study is also defined there.

5.1.3 M3: Distortion Metrics

A *case notion* is an object type we choose as the case. Flattening on type *T* gives one row for every distinct pair of an event and a T-object, the same as the flattening function in `pm4py`. Events with several T-objects are repeated, events with none are dropped, and only event-level attributes survive. A case notion counts as *analysed* when it covers at least 1,000 events, which leaves 38 case notions over the seven logs. Table 5.2 defines the metrics.

Table 5.2: Distortion metrics for a case notion T

Metric	Definition	How to read it
Deficiency	Share of events with no T-object	Events a flat view drops
Convergence share	Share of covered events related to two or more T-objects	Events copied into several cases
Copies per event	Flat rows divided by covered events	Inflation of event counts (1.0 means none)
Divergence share	Share of (case, activity) pairs with two or more events whose other-object sets differ	Repeated events that belong to different objects
Nested child	Another type U whose objects are almost exclusive to one case (95% or more), have their own lifecycle (two or more events) and a fan-out of at least 1.5 per case	Items of an order, for example
Cross-instance adjacency	Among directly-follows pairs in cases with two or more nested children, the share whose two events concern disjoint children	Events adjacent in the flat log but unrelated in the data
Coupling	For an ordered pair of types (T, U), the share of U-objects that touch two or more T-objects	How much the cases share objects
Variants per case	Distinct activity sequences divided by cases	How scattered the flat behaviour looks

The thresholds (95% exclusivity, fan-out of 1.5, 1,000 events) are our own choices. They sit at the top of `distortion.py` and `consequence.py`, and none of them was tuned to improve a result.

5.1.4 M4: Consequence Analysis

Distortion metrics tell us how much a flat view differs. The consequence analysis asks what an analyst would read differently. The reference is the object-centric view: every event counted once, plus the object-centric directly-follows relation, which is the union of consecutive-event pairs over the lifecycle of every object of every type. For each case notion we compare four outputs against that reference, and then we look at the process model an analyst would discover from the flat log.

- C1, activity ranking: Spearman rank correlation of the flat activity counts with (a) the counts of the whole process, each event once, and (b) the counts of the same covered events each counted once, which isolates the effect of duplication. We also report the share of activities visible at all.
- C2, directly-follows edges: duplication of event pairs (flat pairs divided by distinct pairs), rank agreement of edge frequencies with and without duplicates, the largest inflation among the ten most frequent reference edges, and, when nested children exist, the share of flat edge instances whose edge type never occurs inside any child (the

spurious edge mass).

- C3, variants: distinct activity sequences, variants per case, and the share of cases covered by the ten most frequent variants.
- C4, entity ranking: for every other object type with at least 30 objects, the rank correlation between the true number of events per object and the row count that a denormalised flat export of T would give, over the objects that appear in the export at all.
- C5, discovered model: the Inductive Miner with infrequent-behaviour filtering [12] (noise threshold 0.2, as implemented in `pm4py` [15]) runs on each flat log. We score the model on the same flat log with token-based replay fitness [13] and token-based precision [14], and we record the size of the process tree, its *loop share* (the share of its visible activities that sit under a loop operator) and the share of the process's activities that the model contains at all.

Precision replays every prefix of every case, so its cost grows with the square of case length. We compute it only when the squared case lengths add up to at most 20 million, and each case notion gets an hour in total. The Inductive Miner copies its process tree with Python's `deepcopy` while simplifying it, and on cases of several thousand events that nests deeper than Python's default limit of 1,000 calls. Two Age of Empires II case notions failed for this reason in our first runs, so `discovery.py` raises the limit inside the mining process.

Two details matter for fairness. Events are coded by sorted event identifier, so ties in time always break the same way. And the analysis keeps *duplication* (the same event counted several times) apart from *narrowing* (events or activities a case notion never sees). An analyst can undo the first by de-duplicating on the event identifier, but the second cannot be recovered from a flat log.

Some object types would never be chosen as a case notion. A farm in Age of Empires II, for example, takes part in only a handful of activities. To separate this structural narrowing from narrowing that an analyst could plausibly encounter, we also look at the *widest* case notion of each log (the one covering the most events) and at *plausible* case notions, meaning those that cover at least half of the log's events. Both views were introduced after the first results were seen, so we report them over a range of thresholds rather than at 50% only.

Every number is computed on the full data, so the remaining uncertainty is about which events and which items happen to be in a log. `bootstrap.py` estimates it with 200 bootstrap replicates and a fixed seed. For counts of case notions it gives every event a Poisson(1) weight and recomputes coverage and visibility. For rank correlations it resamples the ranked items themselves (activities, or visible entities). An earlier version also used event weights for the correlations and produced intervals that did not contain their own estimates: an entity with a single event receives the same weight in its true count and in its row count, which creates artificial agreement. The unweighted replicate reproduces every published value, and the script checks that before it starts.

5.1.5 M5: Predicate Engine (Appendix-Level)

We keep a rule-based engine from earlier work, `cooc.py` with `registry.py`. It evaluates

eleven natural rules (for example, a change is merged only after an approval vote, or no submission after the assessment deadline) on native, flat and denormalised-flat views of CROP and OULAD. Two logs and eleven rules are too few to support a conclusion, so the engine is excluded from the main results.

5.1.6 M6: Validation

`sql_crosscheck.py` re-implements four CROP rules in plain SQL on the OCEL SQLite file, opened read-only. `check_consequence.py` compares the fast consequence code with a slow plain-Python version on a real slice of the Enron log. `check_distortion.py` does the same for the distortion metrics, using only dictionaries and sets. By default it runs on 6,000-event slices of Enron, Bundestag and Age of Empires II, and with `--full` (part of `make study`) it recomputes every metric on all seven full logs and compares the result with the published files. Finally, the OULAD counts are recomputed independently from the raw CSV files. Section 5.2 reports the results.

5.1.7 M7: Summaries

`summarize_distortion.py`, `summarize_log_level.py`, `summarize_consequence.py` and `summarize_discovery.py` combine the per-log JSON files into the CSV and Markdown tables used in Chapter 7. `make study` runs the main sequence and the full-log check, and `make extended` runs the bootstrap and the process-discovery comparison, which take much longer.

5.2 Testing

We tested the project as we would test an external piece of work. Table 5.3 lists each check, what it establishes and its outcome.

Table 5.3: Verification checks

Check	What it proves	Result
Download integrity	Zenodo files match the published size and MD5	Verified for the Zenodo sources; SHA-256 recorded for every file, including OULAD
Reproducibility	Re-running gives the same outputs	Tracked result files were byte-identical after a re-run (checked with Git)
SQL cross-check	The predicate code matches independent SQL	4 of 4 CROP rules match exactly: 522, 19,109, 164 and 423 flagged changes
Brute-force self-check	The fast consequence code equals plain Python	Directly-follows pairs identical on 4 case notions; entity ranking identical
Distortion self-check	The pandas distortion metrics equal a plain-Python version	Identical on all seven full logs: 87 case notions (every object type) and 294 type pairs
Bootstrap reproduction	The resampling code computes the same statistics as the analysis	The unweighted replicate matches all 38 case notions exactly
Independent recomputation	OULAD counts match the raw CSV files	15,348 enrolments with a late submission, 415 submissions after unregistration, 29 flagged by the stage-order rule: all exact
Bugs found by our own checks	The checks detect real errors	Three bugs found and fixed (see below)

The checks caught three bugs of our own, and we fixed all three before producing the results in this report.

- Duplicate events: the CROP converter wrote duplicate event rows, about 17% of them. They are removed.
- A missed abandon form: the converter did not recognise one phrasing of the Gerrit abandon message ("Patch Set N: Abandoned"). It does now.
- Tie-breaking: events with the same timestamp could be ordered differently in two code paths. Events are now ordered by timestamp and then by event identifier.

Some checks were not carried out. No one outside the team labelled the findings. We reviewed a stratified sample of classified CROP comments by hand and found no further misclassification, but the reviewer was one of us, so it is not an independent label set. The converters are checked by us, not by a second team, and the SQL cross-check tests the predicate code, not the CROP converter. The activity-ranking numbers are recomputed only by the bootstrap code, which loads the data the same way as the analysis it checks.

CHAPTER 6
PROJECT DEMONSTRATION

The live demonstration runs five commands from the `realdata` directory. We ran them for this report and saved the output under `docs/review-3/demo/`, so that the same output can be shown if network or hardware problems occur during the review. Table 6.1 lists the steps and the measured times.

Table 6.1: Demonstration steps

Step	Command	What the panel sees	Time
1	`python3 -I src/fetch_sources.py`	All eight sources already present; nothing is downloaded again	0.1 s
2	`.venv/bin/python -I src/check_consequence.py`	Self-check against the plain-Python version	3.7 s
3	`.venv/bin/python -I src/distortion.py enron`	Distortion table for the Enron log	9.5 s
4	`.venv/bin/python -I src/consequence.py enron`	Rankings, edges, variants and entities for Enron	4.1 s
5	`head -4 results/consequence_by_log.md`	The cross-log summary table	instant

6.1 Step 1: Check that the data is present

Command: python3 -I src/fetch_sources.py

```
[T1e_angular_commits] already present, skipping
[T2a_oulad] already present, skipping
[T2b_moodle_cadiz] already present, skipping
[T3a_crop] already present, skipping
[T4a_bundestag] already present, skipping
[T4b_enron] already present, skipping
[T4c_ethereum] already present, skipping
[T4d_aoe2] already present, skipping
python3 -I src/fetch_sources.py  0.04s user 0.02s system 58% cpu 0.109 total
```

The manifest lists eight sources, and all of them are already on disk with a `SOURCE.md`, so the script skips them. On a clean machine the same command downloads about 350 MB.

6.2 Step 2: Run the self-check

Command: .venv/bin/python -I src/check_consequence.py

```
consequence self-check passed: DF pairs identical for 4 case notions; entity ranking
      identical
.venv/bin/python -I src/check_consequence.py  1.70s user 0.41s system 57% cpu 3.664 total
```

The fast consequence code and a plain-Python version agree on a real Enron slice. The directly-follows pairs are identical for four case notions, and so is the entity ranking.

6.3 Step 3: Distortion of the Enron log

Command: .venv/bin/python -I src/distortion.py enron

```

=== enron: events=96,175 e2o/event=3.498
  flat[EMAILADDRESS:CC] def=0.778 conv=0.667 copies=  2.114 maxActInfl=  3.286 div=0.769
      var/case=0.241 kids=[] CAS=None
  flat[EMAILADDRESS:FROM] def=0.046 conv=0.000 copies=  1.000 maxActInfl=  1.000 div=0.802
      var/case=0.2079 kids=[] CAS=None
```

```
  flat[EMAILADDRESS:TO] def=0.060 conv=0.334 copies=  1.572 maxActInfl=  2.345 div=0.778
      var/case=0.2526 kids=[] CAS=None
  flat[MESSAGEID    ] def=0.401 conv=0.000 copies=  1.000 maxActInfl=  1.000 div=0.715
      var/case=0.0046 kids=[] CAS=None
  coupling EMAILADDRESS:TO objs in >=2 EMAILADDRESS:CC cases: 0.81 (mean 6.369)
  coupling EMAILADDRESS:FROM objs in >=2 EMAILADDRESS:CC cases: 0.79 (mean 5.277)
  coupling EMAILADDRESS:CC objs in >=2 EMAILADDRESS:TO cases: 0.75 (mean 5.878)
  coupling EMAILADDRESS:TO objs in >=2 MESSAGEID cases: 0.69 (mean 11.962)
  coupling    MESSAGEID objs in >=2 EMAILADDRESS:CC cases: 0.67 (mean 2.173)
  coupling EMAILADDRESS:CC objs in >=2 MESSAGEID cases: 0.66 (mean 7.222)
.venv/bin/python -I src/distortion.py enron  9.18s user 0.29s system 99% cpu 9.502 total
```

Four case notions get flattened: the three e-mail roles and the message identifier. Take the copied-in recipients as the case (EMAILADDRESS:CC) and 77.8% of the events drop out, while the remaining events are copied 2.11 times on average. The message identifier drops 40.1% and copies nothing. The coupling lines show how strongly the cases are interlinked: 81% of the "to" addresses take part in events that belong to two or more different "cc" cases (6.4 on average).

6.4 Step 4: Consequences for the Enron log

Command: .venv/bin/python -I src/consequence.py enron

```
=== enron: events=96,175 reference DF pairs=157,258 edge types=48
  EMAILADDRESS:CC            act: visible=0.7778 rho(all)=0.8656 rho(dup)=1.0 | DF:
      dup=1.5675 rho(dup)=0.9844 top10(dup)=9 maxInfl10=0.357 spur=None | var/case=0.241
      top10cov=0.5653 | entity rho min/med=0.0832/0.473
  EMAILADDRESS:FROM          act: visible=1.0 rho(all)=1.0 rho(dup)=1.0 | DF: dup=1.0
      rho(dup)=1.0 top10(dup)=10 maxInfl10=0.669 spur=None | var/case=0.2079 top10cov=0.623
      | entity rho min/med=0.3863/0.962
  EMAILADDRESS:TO            act: visible=1.0 rho(all)=0.9333 rho(dup)=0.9333 | DF:
      dup=1.3065 rho(dup)=0.9792 top10(dup)=9 maxInfl10=1.161 spur=None | var/case=0.2526
      top10cov=0.5419 | entity rho min/med=0.1591/0.887
  MESSAGEID                  act: visible=1.0 rho(all)=1.0 rho(dup)=1.0 | DF: dup=1.0014
      rho(dup)=1.0 top10(dup)=10 maxInfl10=0.035 spur=None | var/case=0.0046 top10cov=0.9917
      | entity rho min/med=0.8327/0.863
.venv/bin/python -I src/consequence.py enron  3.06s user 0.35s system 82% cpu 4.133 total
```

Each line is one case notion. For the copied-recipient notion only 77.8% of the activities are visible, and the activity ranking agrees with the whole process at a rank correlation of 0.87. Measured for duplication alone, the correlation is exactly 1.0, so nearly all of that gap is due to narrowing.

6.5 Step 5: The cross-log summary

Command: head -4 results/consequence_by_log.md

The command prints a Markdown table with one row per log. The lines are too wide for a printed page, so we do not copy them here. The saved output is in `docs/review-3/demo/5_table.txt`, and Table 7.3 in Chapter 7 shows the same numbers.

`make study` re-runs everything. When we ran it for this report it finished without error in 5 minutes 21 seconds and left every tracked result file unchanged (checked with Git).

CHAPTER 7
RESULT AND DISCUSSION

Every number in this chapter comes from the files in `realdata/results` and regenerates with `make study`. These are measurements on seven real public logs. They say nothing about the capstone portal.

7.1 The Seven Logs

Table 7.1: Profile of the seven logs

Log	Domain	Events	Objects	Object types	Objects per event	Analysed case notions
CROP (Gerrit)	Code review	306,716	55,323	3	3.0	3
OULAD	Education	216,532	61,577	4	3.8	4
German Bundestag	Legislation	62,719	37,665	44	1.1	9
Age of Empires II	Game play	34,860	5,054	25	4.3	11
Ethereum (window)	Blockchain	300,000	17,773	5	1.6	5
Angular commits	Software commits	27,842	28,317	2	88.0	2
Enron e-mail	E-mail	96,175	70,541	4	3.5	4

The logs cover code review, education, legislation, e-mail, blockchain and game play, which gives 38 analysed case notions. The Ethereum figures are for a contiguous window of 300,000 events from a log with 3.2 million.

7.2 Distortion by Flattening

Table 7.2: Flattening distortion per log (analysed case notions)

Log	Regime	Median share of events dropped	Median copies per event	Largest copies per event	Notions with over 5% duplication	Type pairs sharing 50% or more
CROP (Gerrit)	Star	0.0%	1.00	1.00	0 of 3	4 of 6
OULAD	Star	0.0%	1.00	1.00	0 of 4	7 of 12
German Bundestag	Partitioned	90.8%	1.00	1.27	2 of 9	2 of 13
Age of Empires II	Partitioned	86.3%	1.01	1.76	4 of 11	50 of 137
Ethereum (window)	Partitioned	68.9%	1.04	1.09	2 of 5	10 of 20
Angular commits	Fan-out	0.1%	44.06	81.65	2 of 2	2 of 2
Enron e-mail	Fan-out	23.1%	1.29	2.11	2 of 4	8 of 12

The logs fall into three groups, shown in Figure 7.1.

In the star logs (CROP and OULAD) every event touches exactly one object of each type. Flattening copies nothing (1.00 copies per event) and, in the median, drops nothing. On these two measures the distortion is zero.

The partitioned logs (Bundestag, Age of Empires II, Ethereum) copy little, at most 1.27, 1.76 and 1.09 copies per event. However, different object types cover different parts of the log, so the median case notion drops between 69% and 91% of the events.

In the fan-out logs (Angular commits and Enron) one event relates to many objects of the same

type. The largest duplication is 81.65 copies per event in Angular (the branch case notion) and 2.11 in Enron.

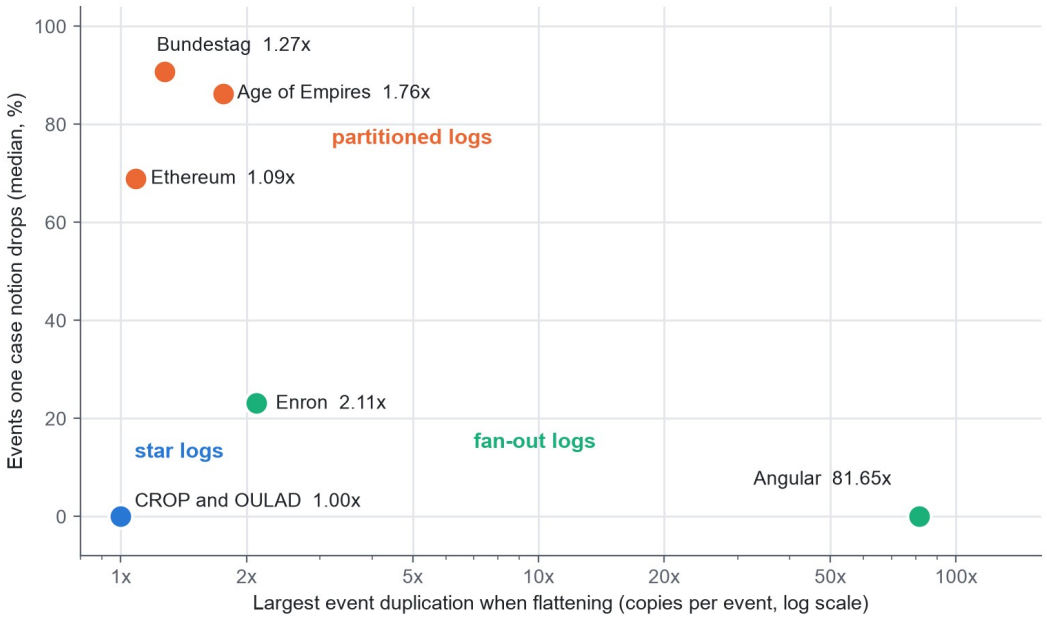


Figure 7.1: Duplication and dropped events for the seven logs, by regime

Coupling adds a second perspective. Figure 7.2 shows the share of object-type pairs where the objects of one type are shared across the cases of the other. In every log except the Bundestag and Age of Empires, at least half the type pairs share more than half of their objects, and in Angular both of the two pairs do.

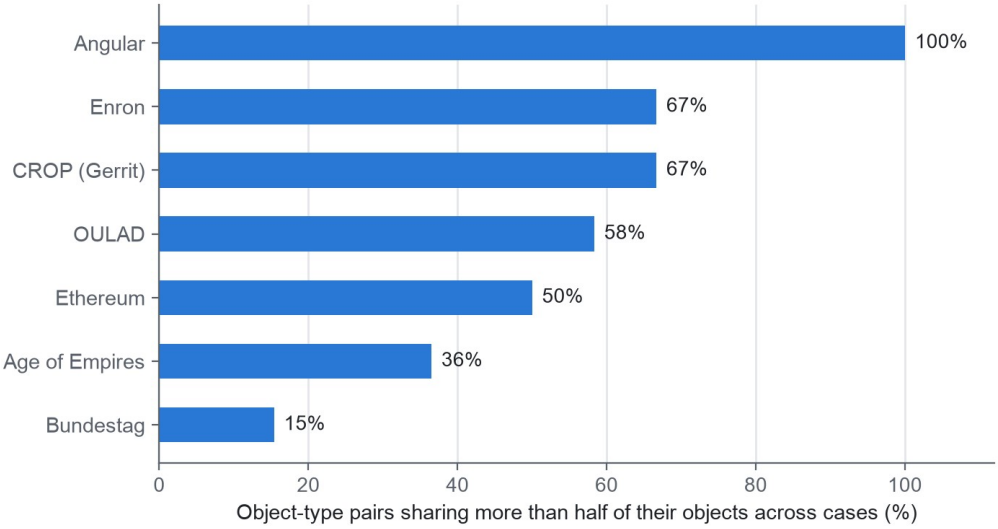


Figure 7.2: Object-type pairs that share more than half of their objects across cases

7.3 Consequences for Standard Outputs

Table 7.3: Consequences of flattening per log

Log	Notions	Activities visible (median)	Activity rank correlation, whole process (median / lowest)	Lowest, duplication only	Largest edge duplication	Lowest entity-rank correlation
CROP (Gerrit)	3	1.00	1.00 / 1.00	1.00	1.00	1.00
OULAD	4	1.00	1.00 / 0.87	1.00	1.00	0.98
German Bundestag	9	0.10	0.17 / 0.00	1.00	1.26	0.34
Age of Empires II	11	0.12	0.21 / -0.05	1.00	1.69	-0.41
Ethereum (window)	5	0.79	0.64 / 0.02	1.00	1.02	-0.01
Angular commits	2	0.98	0.88 / 0.81	0.81	74.05	0.88
Enron e-mail	4	1.00	0.97 / 0.87	0.93	1.57	0.16

Three findings follow from this table.

First, duplication alone rarely changes a ranking. When the activity ranking from flat rows is compared with the ranking of the same events counted once, and only 2 of the 38 case notions fall below a rank correlation of 0.95 (bootstrap interval 1 to 3): Angular files (0.81, interval 0.66 to 0.90) and the Enron "to" role (0.93). The Enron value is less certain: with only nine activities, its interval runs from 0.53 to 1.00. Where a log has no duplication, as in the star logs, the correlation is exactly 1.0.

Second, narrowing is the bigger effect. In 20 of the 38 case notions the flat log shows fewer than half of the activities of the whole process. The activity rank correlation with the whole process has a median of only 0.17 in the Bundestag (interval 0.14 to 0.29) and 0.21 in Age of Empires II (0.15 to 0.29). A flat view of those logs is therefore not a distorted picture of the whole process but a small part of it.

Much of this narrowing is structural. Eighteen of the 20 narrowed case notions come from the Bundestag and Age of Empires II, where an object type such as a written parliamentary question or a farm is in effect a sub-process of its own, and a farm cannot be expected to see the rest of the game. We therefore also examined the case notions an analyst would plausibly choose. Fifteen of them cover at least half of their log, and 2 of those are still narrowed. Villager, in Age of Empires II, covers 52.8% of the events but shows 17% of the activities. OULAD's Assessment case notion covers 80% of the events and shows one activity in three: every submission is present, while every registration and withdrawal is lost. More importantly, two logs have no plausible case notion at all. The widest case notion covers 32.8% of the Bundestag events (and 3% of its 148 activities) and 37.4% of the Ethereum window. In the other five logs the widest case notion covers at least 95% of the events.

These counts do not depend on a single threshold. With the narrowing threshold at 0.3, 0.4, 0.5, 0.6 and 0.7, the number of narrowed case notions is 17, 18, 20, 20 and 20. With the coverage threshold at 30% to 70%, there are 21, 16, 15, 13 and 13 plausible case notions, of which 5, 3, 2, 1 and 1 are narrowed, and OULAD Assessment is narrowed every time. Raising the 1,000-event cut-off to 2,000 or 5,000 leaves 15 of 33 and 10 of 28 narrowed. In all 200 bootstrap replicates the counts remain at 20 narrowed, 15 plausible and 2 both.

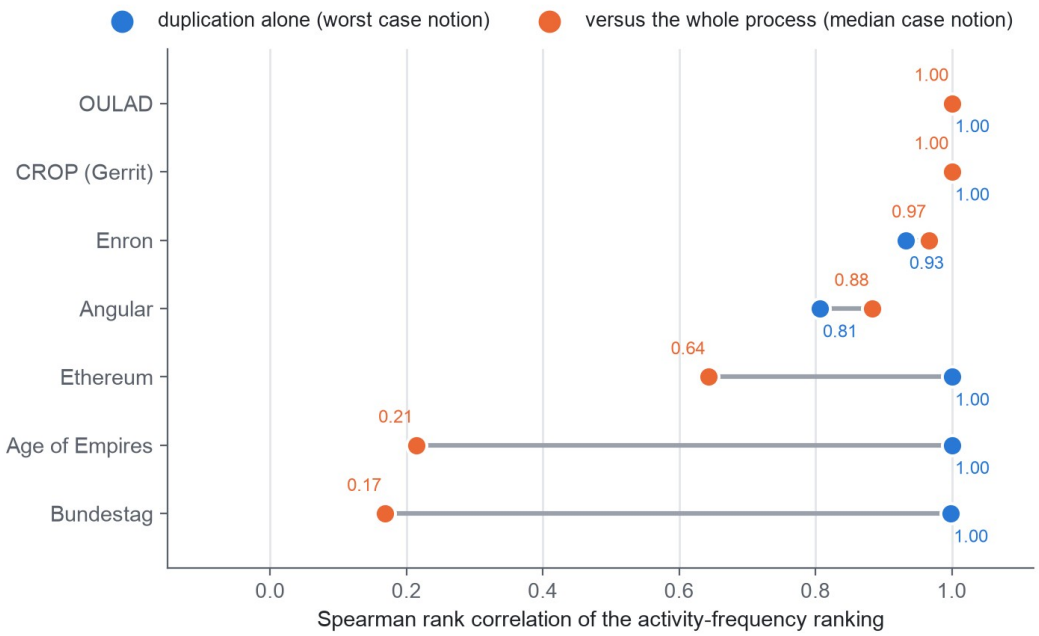


Figure 7.3: Activity ranking agreement: duplication alone versus the whole process

Third, careless counting can mislead. Suppose an analyst ranks entities (people, for example) by counting rows in a denormalised flat export. Among the entities that appear in the export, the worst-case rank correlations with the true number of events per entity are 0.16 in Enron (interval 0.14 to 0.18), -0.01 in Ethereum (-0.03 to 0.02) and -0.41 in Age of Empires II (-0.67 to -0.26). An analyst who de-duplicates on the event identifier would avoid this, so this result is a warning about careless counting rather than an unavoidable loss.

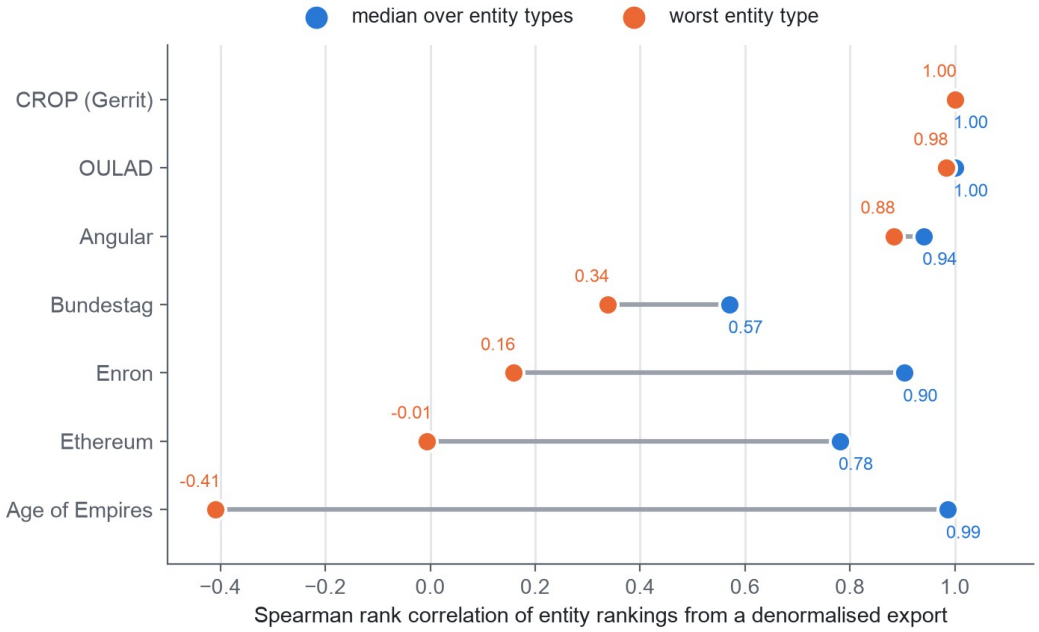


Figure 7.4: Entity ranking agreement from a denormalised flat export

Directly-follows edges show a smaller but measurable effect. Angular has the largest duplication of event pairs (74 copies for the branch case notion). Where nested children exist, some flat edges join events that are adjacent in the flat log but unrelated in the data. These

spurious edges account for at most 27.7% in Age of Empires II and 16.6% in OULAD.

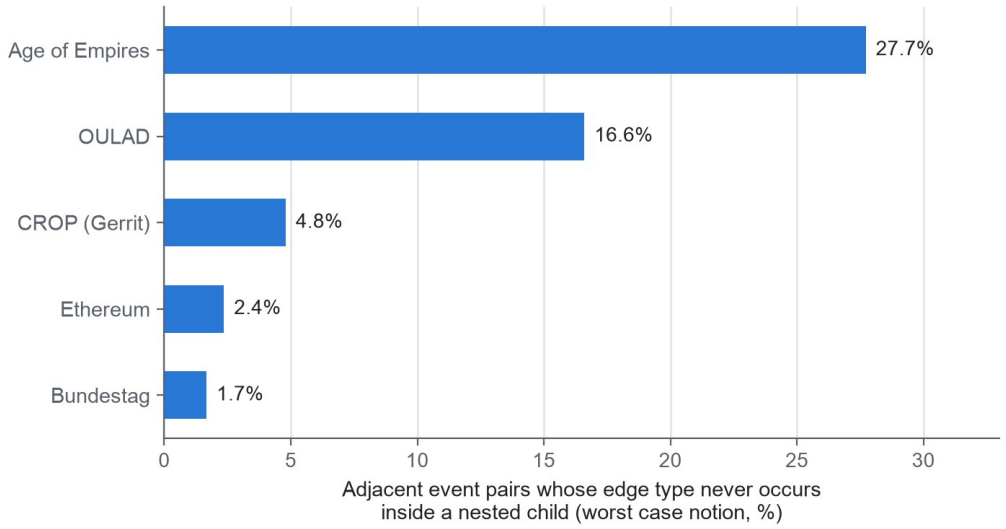


Figure 7.5: Spurious directly-follows edges, worst case notion per log

7.4 Process Discovery on Flat Logs

Process discovery is the most common use of process mining, so we discovered a model from every flat log (C5 in Section 5.1). It finished for 35 of the 38 case notions. The remaining 3 did not finish: Angular commits branches (time budget or failure); Age of Empires II Match (time budget or failure); Age of Empires II Session (time budget or failure). Table 7.4 summarises the results per log.

Table 7.4: Process discovery on the flat logs (Inductive Miner, noise threshold 0.2)

Log	Mined	Lowest fitness	Median precision (notions)	Models with 90% or more of activities in loops	Models with under half of the activities
CROP (Gerrit)	3 of 3	0.975	0.65 (1)	3	0
OULAD	4 of 4	0.980	0.98 (2)	3	1
German Bundestag	9 of 9	0.955	0.79 (9)	1	9
Age of Empires II	9 of 11	0.977	0.30 (9)	2	9
Ethereum (window)	5 of 5	0.979	0.07 (1)	5	1
Angular commits	1 of 2	0.987	0.60 (1)	0	0
Enron e-mail	4 of 4	0.984	0.99 (2)	2	0

First, every model appears to be of high quality. Replay fitness runs from 0.955 to 1.000, with a median of 0.994. All 20 narrowed case notions are in that range, yet 20 of the models contain fewer than half of the process's activities. Fitness is measured against the flat log from which the model was discovered, so it cannot reflect what flattening removed. Figure 7.6 illustrates this: the points on the left represent models of a small fraction of the process, and they score as well as the complete ones.

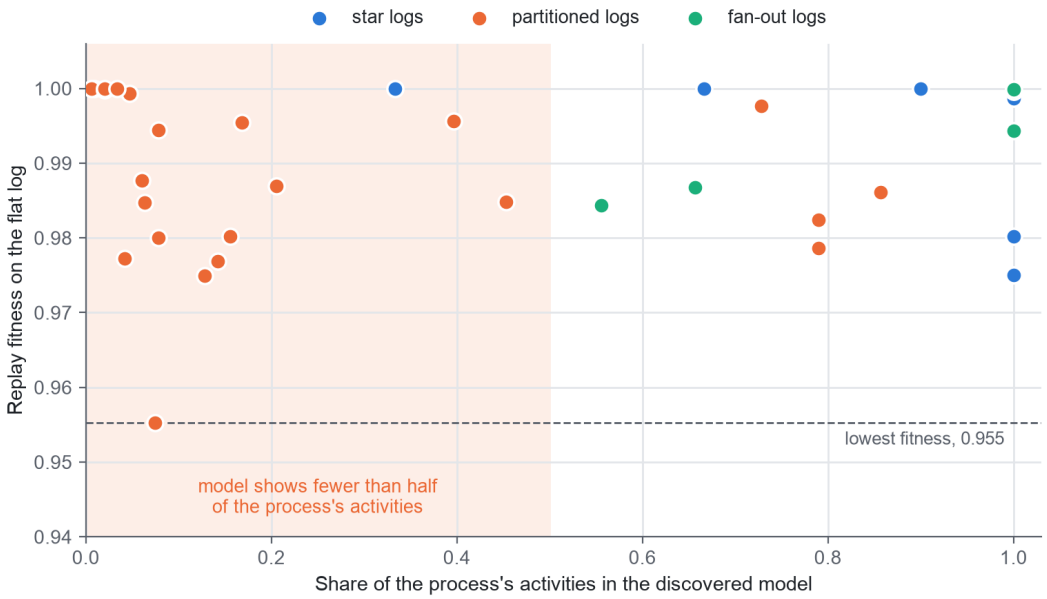


Figure 7.6: Replay fitness against the share of the process's activities in the discovered model, one dot per case notion

Second, divergence appears as loops. When a flat case collects repeated events that belong to different objects, the miner tends to place them inside a loop. Across case notions the loop share rises with the divergence share (Spearman rank correlation 0.66). At least 90% of the activities sit under a loop in 12 of the 17 case notions with a divergence share of 0.5 or more, and in only 4 of the rest. Precision, which could be computed for 25 case notions, shows the consequence. Its median is 0.67, and the lowest values (0.04 for Age of Empires II player, 0.07 for Ethereum orders (ORDER), 0.09 for German Bundestag legislation (Gesetzgebung)) all come from models in which nearly every activity is inside a loop, so the models accept almost any order of events.

Given the number of case notions and the seven logs, we interpret these correlations as tendencies rather than general laws.

7.5 Discussion

Our initial expectation was that flattening mainly duplicates events. The data did not support this. Duplication is large in only two of the seven logs. The common pattern is that a case notion sees only part of the process, and the flat log gives no indication of what is missing. The usual quality check does not reveal this: a model discovered from a narrowed flat log still replays that log almost perfectly, so fitness remains high precisely when the view is most partial.

For the capstone, only a hypothesis can be stated, since no portal data is available. The portal's documented audit log relates each action to one project and to the person who acted. If real data has that shape, flattening on the project would copy few events, but it would hide how one guide's reviews spread across many projects, and it would hide the people. That is the narrowing pattern, not the duplication pattern. The next step is to test this on a de-identified portal log. Until then we make no claim about the portal.

7.5.1 Threats to Validity

- Seven logs from several domains: the three regimes are descriptive, and we do not claim that they are exhaustive.
- Our choices: case notions are object types (and roles for Enron), and the thresholds for nested children and for the minimum case-notion size are ours, written in the code.
- A window of Ethereum: those results come from a contiguous window of 300,000 events.
- A data-quality flag in Enron: about 0.1% of the events (72) have timestamps before 1990, which is unusual for a corpus from 1998 to 2002. They were retained.
- Checks by us: the independent implementations are independent of the code under test, not of the team, and no one outside the team labelled the findings.
- One miner: the discovery comparison uses one algorithm with one noise threshold, and other miners may well give different models.

7.6 Cost Analysis

By cost we mean the money and effort to produce or reproduce the study. Table 7.5 summarises it.

Table 7.5: Cost of producing and reproducing the study

Item	Producing the study	Reproducing it
Data	Nil (open data, CC BY 4.0)	About 350 MB download for the core set
Software	Nil (open-source libraries; `pm4py` is AGPL-3.0)	Python 3.14 and the pinned packages in `requirements.txt`
Hardware	A laptop the team already owned (8 GiB memory)	Any laptop with 8 GiB of memory or more
Cloud or paid services	None	None
Run time	Seconds to minutes per log	About 4 to 10 seconds for the Enron log; `make study` for all seven logs took 5 minutes 21 seconds

The main cost is the team's time, much of it spent on verification. The monetary cost is nil.

CHAPTER 8
CONCLUSION

We set out to study capstone review processes with OCEL 2.0. Portal logs were not available, so the project turned into two things: a documented OCEL 2.0 design of the capstone lifecycle, and a measurement of the effects of flattening on seven real public logs. The main conclusions are as follows.

First, a reproducible and verified pipeline has been built. It downloads and verifies seven public logs, converts two of them to OCEL 2.0, computes distortion metrics for 38 case notions and compares standard outputs between flat and object-centric views. A second, plain-Python implementation reproduces the distortion metrics exactly on all seven full logs, other independent checks agree with the parts they cover, and the checks caught three bugs of our own, which we fixed.

Second, flattening does not mainly duplicate events. Duplication is large only in the fan-out logs, up to 81.65 copies of an event in Angular, and only 2 of 38 case notions fall below a rank correlation of 0.95 because of duplication alone.

Third, the larger effect is narrowing. In 20 of 38 case notions the flat log shows fewer than half of the process's activities. Most of that comes from the structure of the logs, but 2 of the 15 case notions that cover at least half of their log are narrowed too, and in two logs no case notion covers half of the events. The counts hold across every threshold we tried and in every bootstrap replicate.

Fourth, the usual quality checks do not detect narrowing. A model discovered from a narrowed flat log still replays that log with a fitness of 0.955 or more, and divergence turns into loops that accept almost any order of events.

Fifth, careless counting from a flat export can be seriously misleading. Worst-case entity rankings reach a rank correlation of 0.16 (Enron), -0.01 (Ethereum) and -0.41 (Age of Empires II), and none of the bootstrap intervals exceeds 0.2.

Sixth, the capstone model remains a design. We could not test it on portal data, and we claim nothing about how real capstone teams behave.

The measurement part of this work is written up as a journal paper, *What Does Flattening Hide? Measuring Case-Notion Distortion and Its Consequences on Seven Real Object-Centric Event Logs*, which we submitted to the Journal of Intelligent Information Systems (Springer) in October 2026. It is under review.

8.1 Limitations

We have seven logs from five domains, and one of them is a window. The thresholds are ours, and the idea of a plausible case notion came after we had seen the first results. The checks are independent of the code under test but not of the team, no one outside the team has labelled the findings, and the activity-ranking numbers are re-computed only by the bootstrap code. The discovery comparison uses one miner. The rule-based engine covers eleven rules on two logs, and we use it for no conclusion.

8.2 Future Work

- Get independent two-rater labelling of the real-data findings.
- Compare predictive monitoring between flat and object-centric logs, and repeat the discovery comparison with other miners.

REFERENCES

The references follow the IEEE numeric style, numbered in the order in which they are first cited.

[1] W. M. P. van der Aalst, "Object-centric process mining: Dealing with divergence and convergence in event data," in *Software Engineering and Formal Methods (SEFM)*, Lecture Notes in Computer Science. Springer, 2019, pp. 3-25.

[2] A. Berti *et al.*, "OCEL (Object-Centric Event Log) 2.0 specification," arXiv:2403.01975, 2024.

[3] J. N. Adams, G. Park, and W. M. P. van der Aalst, "Preserving complex object-centric graph structures to improve machine learning tasks in process mining," *Engineering Applications of Artificial Intelligence*, vol. 125, p. 106764, 2023.

[4] J. N. Adams and W. M. P. van der Aalst, "Addressing convergence, divergence, and deficiency issues," Lecture Notes in Business Information Processing. Springer, 2024.

[5] P. Waibel, L. Pfahlsberger, K. Revoredo, and J. Mendling, "Causal process mining from relational databases with domain knowledge," arXiv:2202.08314, 2022.

[6] M. Kabierski *et al.*, "Data quality in object-centric event data: Issues classification and evaluation," *Process Science*, vol. 3, 2026.

[7] N. Miri, S. Khayatbashi, J. Zdravkovic, and A. Jalali, "An object-centric approach to multi-entity educational process mining," *Decision Analytics Journal*, vol. 19, p. 100724, 2026.

[8] J. Kuzilek, M. Hlosta, and Z. Zdrahal, "Open University Learning Analytics dataset," *Scientific Data*, vol. 4, p. 170171, 2017.

[9] M. Paixao, J. Krinke, D. Han, and M. Harman, "CROP: Linking code reviews to source code changes," in *Proc. Mining Software Repositories (MSR)*, 2018.

[10] H. Hassani *et al.*, "Supervision of undergraduate final year projects in computing: A case study," *Education Sciences*, vol. 8, no. 4, p. 210, 2018.

[11] W. Premchaiswadi and P. Porouhan, "Process modeling and bottleneck mining in online peer-review systems," *SpringerPlus*, vol. 4, p. 441, 2015.

[12] S. J. J. Leemans, D. Fahland, and W. M. P. van der Aalst, "Discovering block-structured process models from event logs containing infrequent behaviour," in *Business Process Management Workshops (BPM 2013)*, Lecture Notes in Business Information Processing, vol. 171. Springer, 2014, pp. 66-78.

[13] A. Berti and W. M. P. van der Aalst, "Reviving token-based replay: Increasing speed while improving diagnostics," in *Proc. Algorithms and Theories for the Analysis of Event Data (ATAED 2019)*, CEUR Workshop Proceedings, vol. 2371, 2019, pp. 87-103.

[14] J. Munoz-Gama and J. Carmona, "A fresh look at precision in process conformance," in *Business Process Management (BPM 2010)*, Lecture Notes in Computer Science, vol. 6336. Springer, 2010, pp. 211-226.

[15] A. Berti, S. van Zelst, and D. Schuster, "PM4Py: A process mining library for Python," *Software Impacts*, vol. 17, p. 100556, 2023.

[16] Datasets used (all CC BY 4.0): German Bundestag, Zenodo record 16811928 (H. Diers, V. Peeva, C. Rennert, W. M. P. van der Aalst); Enron e-mail OCEL, Zenodo 21989732 (A. Berti); Ethereum execution data, Zenodo 21989580 (A. Berti); Age of Empires II, Zenodo 13365584 (L. Liss, N. Elbert, C. M. Flath); Angular commits, Zenodo 8430332 (M. Pegoraro); CROP, Zenodo 3599150 (M. Paixao, J. Krinke, D. Han, M. Harman); OULAD, UCI Machine Learning Repository, doi:10.24432/C5KK69.

APPENDIX A - SAMPLE CODE

The listings are copied from the repository by a script, so they are the code that produced the results. Long lines are wrapped for printing, and file names are relative to the `realdata/src` *directory.*

A.1 Case-centric flattening (ocel_io.py)

```
def flatten(ocel: Ocel, case_type: str) -> pd.DataFrame:
    """Case-centric flattening: one row per (event, object of ``case_type``).

    Mirrors pm4py's ``ocel_flattening``: events related to several objects of the
    case type are repeated (convergence); events with none are dropped (deficiency);
    only event-level attributes survive, never attributes or identities of other
            objects.
    """
    rel = ocel.relations[ocel.relations["otype"] == case_type][["eid",
            "oid"]].drop_duplicates()
    flat = rel.merge(ocel.events, on="eid", how="left").rename(columns={"oid":
            "case_id"})
    return flat.sort_values(["case_id", "timestamp", "eid"],
            kind="stable").reset_index(drop=True)
```

A.2 Deficiency, convergence and divergence (ocel_io.py)

```
def flattening_stats(ocel: Ocel, case_type: str) -> dict:
    """Deficiency, convergence and divergence for one case notion."""
    n_events = len(ocel.events)
    rel_t = ocel.relations[ocel.relations["otype"] == case_type][["eid",
            "oid"]].drop_duplicates()
    covered = rel_t["eid"].nunique()
    per_event = rel_t.groupby("eid")["oid"].nunique()
    stats = {
        "case_type": case_type,
        "cases": int(rel_t["oid"].nunique()),
        "events": n_events,
        "flat_rows": int(len(rel_t)),
        "row_multiplier": round(len(rel_t) / n_events, 3) if n_events else None,
        "deficiency_share": round(1 - covered / n_events, 4) if n_events else None,
        "convergence_share": round(float((per_event > 1).mean()), 4) if covered else
                0.0,
    }
    # Divergence: within a case, the same activity occurs >= 2 times and the repeated
    # events differ in the set of objects of at least one other type.
    other = ocel.relations[ocel.relations["otype"] != case_type][["eid", "otype",
            "oid"]].drop_duplicates()
    if other.empty or rel_t.empty:
        stats["divergence_share"] = 0.0
        return stats
    act = ocel.events[["eid", "activity"]]
    case_events = rel_t.merge(act, on="eid")
    reps = case_events.groupby(["oid", "activity"])["eid"].transform("count") > 1
    case_events = case_events[reps]
    if case_events.empty:
        stats["divergence_share"] = 0.0
        return stats
    total_pairs = case_events[["oid", "activity"]].drop_duplicates().shape[0]
    sig = (other.sort_values(["eid", "otype", "oid"]).groupby(["eid",
            "otype"])["oid"].agg(lambda s: "|".join(s)).reset_index())
    merged = case_events.merge(sig, on="eid", how="left")
    diverging = merged.groupby(["oid_x", "activity", "otype"])["oid_y"].nunique()
    diverging_pairs = diverging[diverging > 1].reset_index()[["oid_x",
            "activity"]].drop_duplicates().shape[0]
    stats["divergence_share"] = round(diverging_pairs / total_pairs, 4)
    return stats
```

A.3 Rank agreement helpers (consequence.py)

```
def spearman(a: np.ndarray, b: np.ndarray) -> float | None:
    if len(a) < 3 or np.ptp(a) == 0 or np.ptp(b) == 0:
        return None
    return round(float(spearmanr(a, b).statistic), 4)
```

```
def topk_overlap(ref: np.ndarray, other: np.ndarray, k: int) -> int:
    """Size of the intersection of the top-k index sets (ties broken by index,
           deterministic)."""
    k = min(k, len(ref))
    top = lambda x: set(np.lexsort((np.arange(len(x)), -x))[:k])  # noqa: E731
    return len(top(ref) & top(other))

def variant_summary(seqs: pd.Series) -> dict:
    vc = seqs.value_counts()
    return {"cases": int(len(seqs)), "variants": int(len(vc)), "variants_per_case":
           round(len(vc) / len(seqs), 4),
           "top10_variant_coverage": round(float(vc.head(10).sum() / len(seqs)), 4)}
```

A.4 Integrity check on a Zenodo download (fetch_sources.py)

```
def fetch_zenodo(src: dict, out_dir: Path) -> list[dict]:
    rec = get_json(f"https://zenodo.org/api/records/{src['record']}")
    index = {f["key"]: f for f in rec.get("files", [])}
    rows = []
    for name in src["files"]:
        meta = index.get(name)
        if meta is None:
            raise SystemExit(f"{src['id']}: {name} not listed in record
                   {src['record']}")
        url = meta.get("links", {}).get("self") or f"https://zenodo.org/api/records/{src
               ['record']}/files/{urllib.parse.quote(name)}/content"
        dest = out_dir / name
        print(f"[{src['id']}] {name} ({meta['size'] / 1e6:.1f} MB) ...", flush=True)
        md5, sha, size = stream(url, dest)
        expected = meta.get("checksum", "").split(":", 1)[-1]
        if size != meta["size"] or (expected and md5 != expected):
            dest.unlink(missing_ok=True)
            raise SystemExit(f"{src['id']}: {name} failed integrity check (size {size}
                   vs {meta['size']}, md5 {md5} vs {expected})")
        rows.append({"file": name, "bytes": size, "sha256": sha, "md5_verified":
               bool(expected)})
    return rows
```

A.5 Safe extraction of archives (extract_archives.py)

```
def main() -> None:
    for archive, (target, skip) in JOBS.items():
        src, dst = RAW / archive, RAW / target
        if not src.exists():
            print(f"missing: {archive}")
            continue
        if dst.exists():
            print(f"exists, skipping: {target}")
            continue
        with zipfile.ZipFile(src) as z:
            bad = [i.filename for i in z.infolist() if i.filename.startswith("/") or
                   ".." in i.filename.split("/")]
            if bad:
                raise SystemExit(f"{archive}: unsafe member paths {bad[:3]}")
            dst.mkdir(parents=True)
            for info in z.infolist():
                if info.filename not in skip:
                    z.extract(info, dst)
        print(f"extracted {archive} -> {target}")
```